

Clase 3 - Sistemas Distribuidos

Capítulo 2: Arquitecturas
(Tanenbaum & Van Steen, 4ta ed.)

Cap 2. Arq en SD

2 Architectures	55
2.1 Architectural styles	56
2.1.1 Layered architectures	57
2.1.2 Service-oriented architectures	62
2.1.3 Publish-subscribe architectures	68
2.2 Middleware and distributed systems	73
2.2.1 Middleware organization	74
2.2.2 Modifiable middleware	78
2.3 Layered-system architectures	78
2.3.1 Simple client-server architecture	79
2.3.2 Multitiered Architectures	80
2.3.3 Example: The Network File System	83
2.3.4 Example: The Web	85
2.4 Symmetrically distributed system architectures	88
2.4.1 Structured peer-to-peer systems	90
2.4.2 Unstructured peer-to-peer systems	92
2.4.3 Hierarchically organized peer-to-peer networks	95
2.4.4 Example: BitTorrent	96
2.5 Hybrid system architectures	98
2.5.1 Cloud computing	98
2.5.2 The edge-cloud architecture	100
2.5.3 Blockchain architectures	104
2.6 Summary	108

Que es una Arquitectura??

- Una arquitectura es “la organización fundamental de un sistema definido por...”
 - Sus **componentes** reemplazables... (building blocks)
 - Las relaciones entre los componentes (conectores e interfaces, dependencias) y su entorno...
 - Los principios guiando su diseño (reglas y restricciones para construir los bloques y conectores) y evolución...

Que es una Arquitectura??

- Una arquitectura es “la organización fundamental de un sistema definido por...”
 - Sus **componentes** reemplazables... (building blocks)
Organización lógica y física
 - Las relaciones entre los componentes (conectores e interfaces, dependencias) y su entorno...
cómo interactúan y se comunican
 - Los principios guiando su diseño (reglas y restricciones para construir los bloques y conectores) y evolución...
Facilita manejo de complejidad y escalabilidad

Que es una Arquitectura??

- Una arquitectura es “la organización fundamental de un sistema definido por...”

- Sus **componentes** reemplazables... (building blocks)

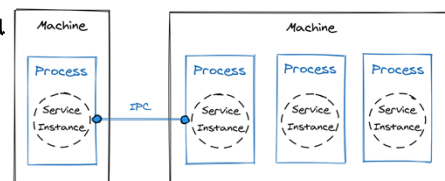
Organización lógica y física

- Las relaciones entre los componentes (conectores e interfaces, dependencias) y su entorno...

cómo interactúan y se comunican

- Los principios guiando su diseño (reglas y restricciones para construir los bloques y conectores) y evolu

Facilita manejo de complejidad y escalabilidad



Propiedades estructurales del sistema

La arquitectura define **las propiedades estructurales del sistema**:

- acoplamiento
- puntos únicos de fallo (SPOF)
- Escalabilidad
- Comunicación

las cuales **no pueden corregirse solo optimizando código**. (no solo implementación)

Importancia de la Arquitectura en SD

- Organización lógica y física de componentes distribuidos
- Define cómo interactúan y se comunican los procesos
- Facilita manejo de complejidad y escalabilidad

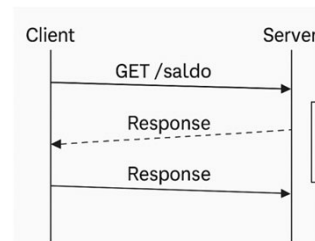
Arquitectura Cliente-Servidor

- Cliente solicita servicios, servidor los procesa y responde
- Protocolos: request-reply, conexión confiable o no confiable
- Ejemplo: NFS, HTTP clásico

¿Qué caracteriza a una arquitectura **Cliente/Servidor** desde el punto de vista arquitectónico (no tecnológico)?

Arquitectura Cliente-Servidor

- Cliente solicita servicios, servidor los procesa y responde
- Protocolos: request-reply, conexión confiable o no confiable
- Ejemplo: NFS, HTTP clásico



Estilos de Arquitectura

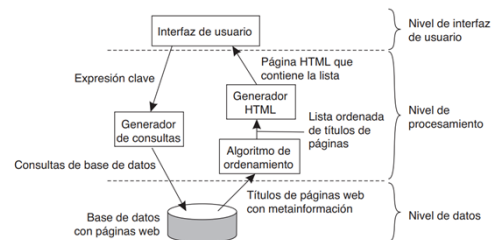
- Arquitectura por Capas
- Arquitectura Orientada a Servicios (SOA y Microservicios)
- Arquitectura Publish-Subscribe
- Arquitectura RESTful y basada en recursos

Arquitectura por Capas

- Organiza componentes en capas lógicas (UI, lógica, datos)
- Ejemplo: Pila de protocolos TCP/IP
- Facilita modularidad y reemplazo de componentes

Arquitectura por Capas

- Organiza componentes en capas lógicas (UI, lógica, datos)
- Ejemplo: Pila de protocolos TCP/IP
- Facilita modularidad y reemplazo de componentes



SOA y Microservicios

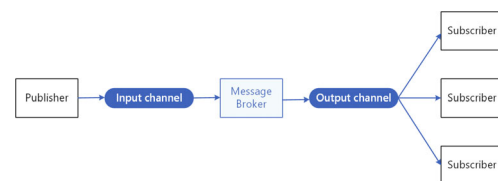
- SOA: Composición de servicios independientes
- Microservicios: Servicios pequeños y desplegables por separado
- Promueven escalabilidad y mantenimiento

Arquitectura Publish-Subscribe

- Procesos publican y se suscriben a eventos sin conocerse entre sí
- Desacoplamiento temporal y referencial
- Ideal para sistemas escalables y dinámicos

Arquitectura Publish-Subscribe

- Procesos publican y se suscriben a eventos sin conocerse entre sí
- Desacoplamiento temporal y referencial
- Ideal para sistemas escalables y dinámicos



Middleware en Sistemas Distribuidos

- Capa entre el SO y las aplicaciones distribuidas
- Ofrece comunicación, seguridad, tolerancia a fallos
- Facilita transparencia y portabilidad

Patrones de Middleware

- Wrappers/Adaptadores: integran componentes heterogéneos
- Interceptors: modifican el flujo para agregar comportamiento
- Middleware modificable para adaptación en tiempo real

Arquitectura Multinivel (n-Tier)

- 2-Tier: Cliente + Servidor
- 3-Tier: Cliente + Servidor de Aplicaciones + BD
- Web y apps modernas usan múltiples capas y microservicios

Ejemplos de Arquitecturas en la Práctica

- NFS: Cliente-servidor para archivos
- Web: Evolución de 2-tier a 3-tier
- Cloud: RESTful y serverless

Cierre y Próxima Clase

- Resumen de arquitecturas y patrones de diseño
- Importancia del middleware y estilos combinados
- Próxima clase: Comunicación en SD (Capítulo 3)